

第 0 週：語法入門 + 暖身

如果你是寫軟體出身 (Python / C / Java)，先讀 [SYNTAX.md](#)。那份用 Python 對照講完 Verilog 的基本語法，看完再進第 1 週會順很多。

本週指令

雙擊 `env.bat` 開好環境 (提示字元要有 `[OSS CAD Suite]`)，然後：

```
cd week00_warmup
```

語法練習場 (配合 [SYNTAX.md](#) 看)：

```
iverilog -o sim.out 02_syntax.v 02_syntax_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

4-bit 計數器：

```
iverilog -o sim.out 01_counter.v 01_counter_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave wave.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_counter` — 4 位元計數器
 - `02_syntax` — 語法練習場 (`{}`、`for`、數字寫法)
-

這裡有什麼

檔案	是什麼
<code>SYNTAX.md</code>	★ 給軟體出身的人的語法入門 (Python 對照)
<code>02_syntax.v</code> / <code>_tb.v</code>	語法練習場： <code>{}</code> 、 <code>for</code> 、 數字寫法都跑一次給你看
<code>01_counter.v</code>	4 位元計數器 (設計)
<code>01_counter_tb.v</code>	它的測試平台
<code>counter.v</code> / <code>tb_counter.v</code>	同樣的內容，原始檔名 (留著給你對照)
<code>wave.gtkw</code>	舊的波形設定

`01_counter.v` 和 `counter.v` 內容一樣 —— 改名是為了讓 `sim` 指令能認得 (`sim` 要求設計檔叫 名稱.v 、測試檔叫 名稱_tb.v) 。

這個設計在做什麼

```
always @(posedge clk) begin
    if (reset)
        count <= 4'b0000;    // reset 時歸零
    else
        count <= count + 1;    // 否則每個時脈加 1
end
```

跑起來會看到：

```
時間=0    | 重置=1 | 計數值=xxxx ( x )
時間=5    | 重置=1 | 計數值=0000 ( 0 )
時間=15   | 重置=0 | 計數值=0001 ( 1 )
時間=25   | 重置=0 | 計數值=0010 ( 2 )
...
```

波形上要看的三件事：

1. `reset` 高的時候，`count` 停在 0
2. `reset` 放開後，`count` 一格一格往上跳
3. `count` 只在 `clk` 上升緣改變

最前面那小段 `x`（紅色）是正反器還沒被寫過的未知狀態，正常現象。

逐行講解

這個設計的**每一行**、以及它跟波形圖的對應關係，寫在 [../explain.md](#)。第一次覺得看不懂就去翻那份。

兩個已知的小問題（之後幾週會解決）

問題	在哪一週解決
沒寫 <code>`timescale</code> ，所以波形時間軸單位是「秒」	第 5 週
<code>#15 reset = 0</code> 剛好撞上時脈邊緣，有競態	第 7 週

現在不用管，知道有這回事就好。

建議順序

1. 讀 [SYNTAX.md](#) —— 語法對照，看不懂就回來翻
2. 跑 `02_syntax` —— 上面講的語法全部實際跑一次
3. 跑 `01_counter` —— 看波形，配合 [../explain.md](#) 的逐行講解
4. 進第 1 週：

```
cd ..  
cd week01_basics
```

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔（`sim.bat`），不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。